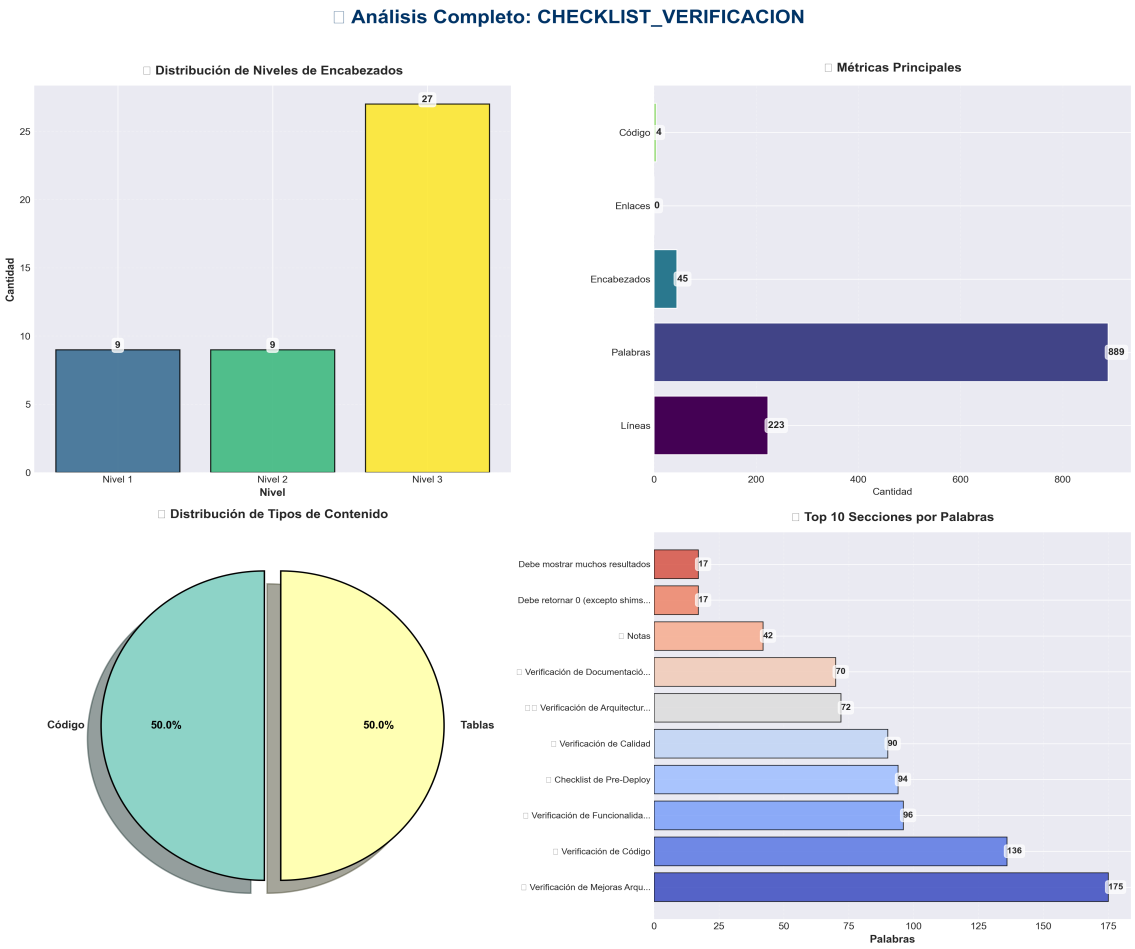


Checklist Verificacion

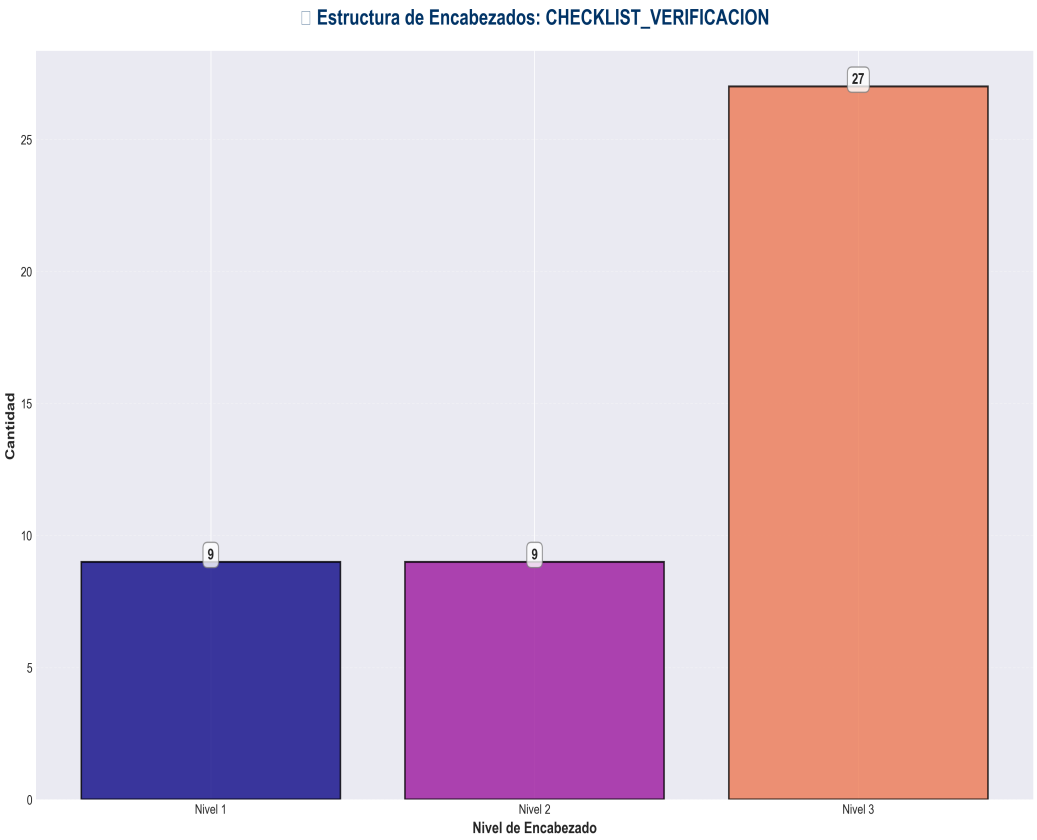
Análisis y Documentación Completa
 Generado el: 25 de November de 2025 a las 11:10

Análisis Visual Completo

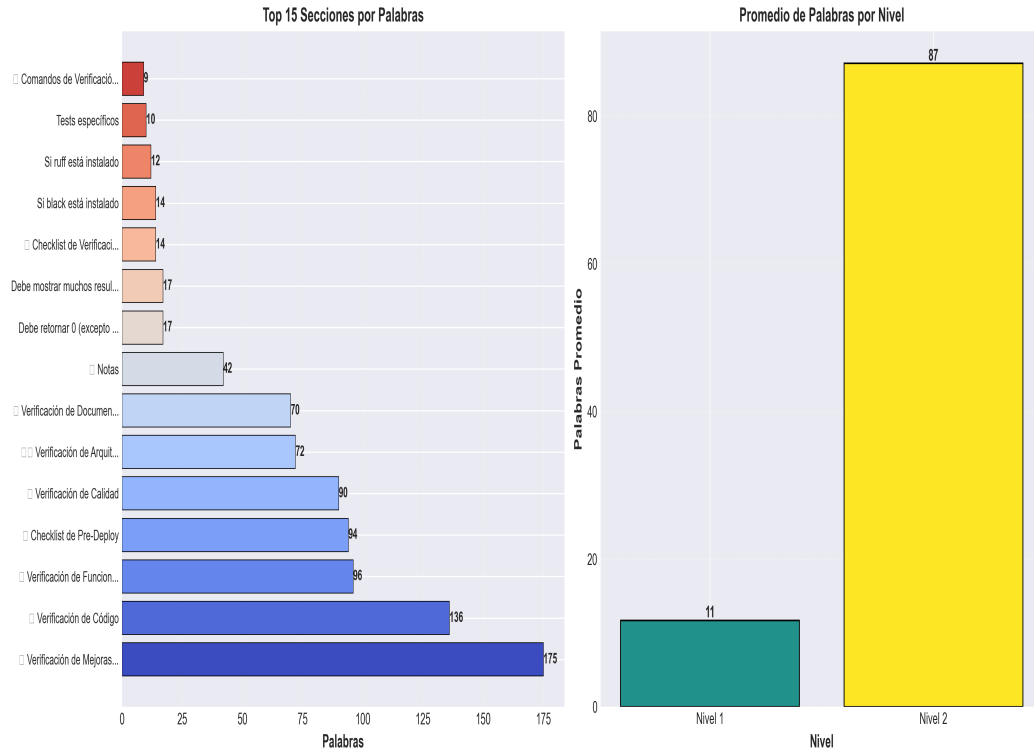
Dashboard Principal



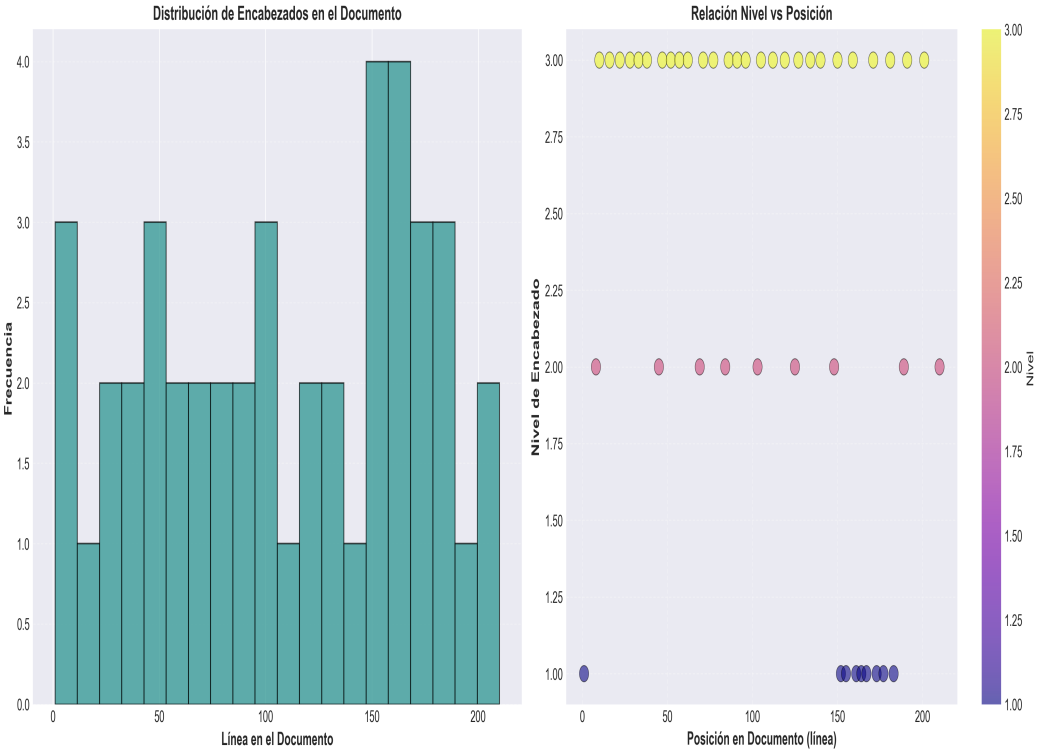
Análisis Adicionales



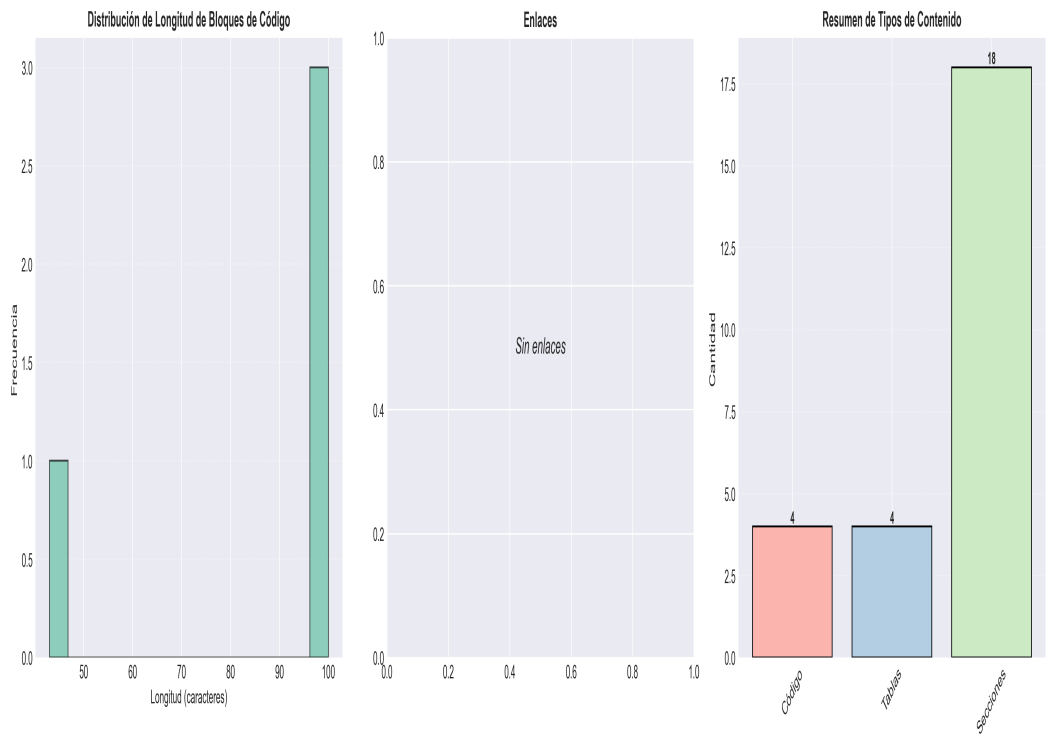
☐ Análisis de Complejidad: CHECKLIST_VERIFICACION



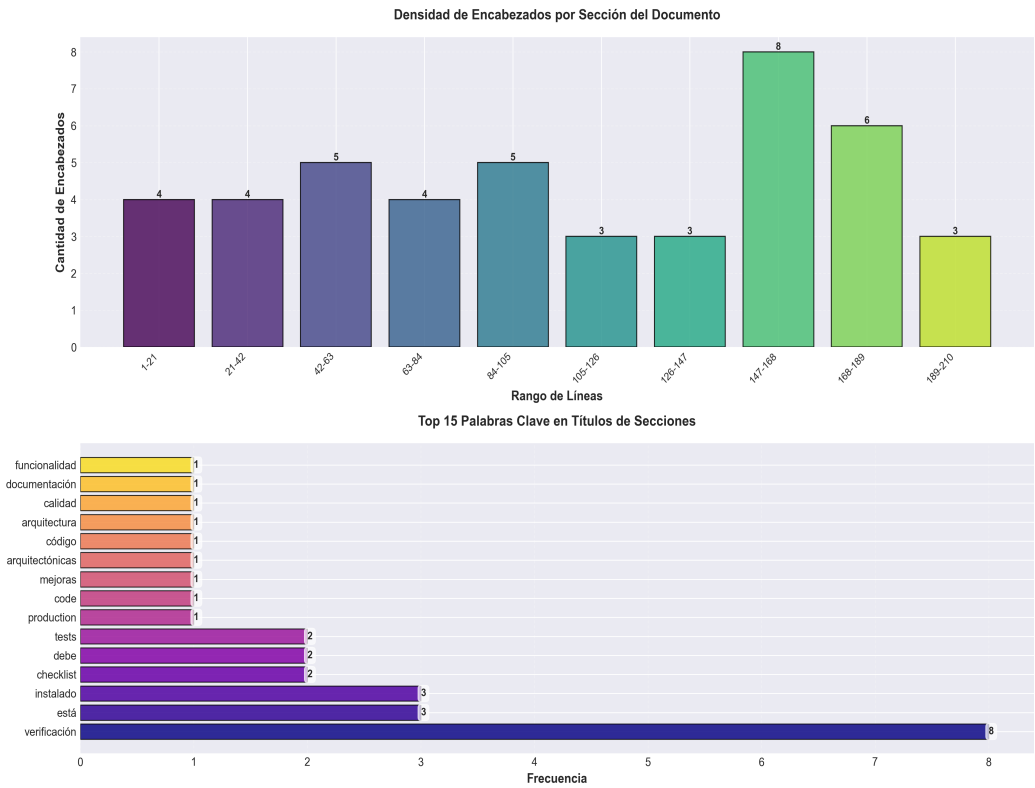
□ Análisis Temporal y Distribución: CHECKLIST_VERIFICACION



□ Análisis Detallado de Contenido: CHECKLIST_VERIFICACION



☐ **Análisis de Densidad y Frecuencia: CHECKLIST_VERIFICACION**



Resumen Ejecutivo: Este documento contiene 889 palabras distribuidas en 223 líneas, con 45 encabezados organizados en 18 secciones principales.

Estadísticas del Documento

Métrica	Valor
Total de líneas	223
Total de palabras	889
Total de caracteres	5,814
Encabezados	45
Bloques de código	4
Tablas	4
Enlaces	0
Imágenes	0
Secciones principales	18

Índice de Secciones

Nivel	Título	Línea
1	■ Checklist de Verificación - Production Code	1
2	■ Verificación de Mejoras Arquitectónicas	8
2	■ Verificación de Código	45
2	■■ Verificación de Arquitectura	69
2	■ Verificación de Calidad	84
2	■ Verificación de Documentación	103
2	■ Verificación de Funcionalidad	125
2	■ Comandos de Verificación	148
1	Debe retornar 0 (excepto shims deprecated)	152
1	Debe mostrar muchos resultados	155
1	Ejecutar todos los tests	161
1	Con cobertura	164
1	Tests específicos	167
1	Si ruff está instalado	173
1	Si black está instalado	177
1	Si mypy está instalado	183
2	■ Checklist de Pre-Deploy	189
2	■ Notas	210

Resumen del Contenido

■ Checklist de Verificación - Production Code

****Versión**:** 2.0.0

****Última actualización**:** 2025-01-27

■ Verificación de Mejoras Arquitectónicas

Phase 1: API Utils Consolidation ■

- [x] `api/api\_utils.py` contiene todas las funciones
- [x] `api\_utils.py` (root) es shim de deprecación
- [x] Todos los imports actualizados
- [x] Tests actualizados

Phase 2: API Entry Points ■

- [x] `api\_unified.py` convertido a shim
- [x] Todas las rutas en `api/routes/`
- [x] `application.py` es factory principal
- [x] `api\_server.py` usa nueva arquitectura

Phase 3: Import Standardization ■

- [x] Todos los imports usan rutas de módulo
- [x] 0 imports root-level (excepto shims)
- [x] `core.config\_manager` usado en todos lados
- [x] `api.middleware` usado en todos lados

Phase 4: Config Manager ■

- [x] `core/config\_manager.py` consolidado
- [x] `config\_manager.py` (root) es shim
- [x] Todos los imports actualizados

Phase 5: Directory Structure ■

- [x] ``code_modules/`` renombrado (sin conflictos)
- [x] Archivos root-level organizados
- [x] Estructura alineada con arquitectura

Phase 6: Architecture Alignment ■

- [x] Application factory actualizado
- [x] ServiceContainer usando imports correctos
- [x] Cumplimiento de capas verificado

■ Verificación de Código

Imports

- [] Ejecutar: ``grep -r "from (api_utils|config_manager|api_auth|api_middlewares) import" --include="*.py"`` → Debe retornar 0 (excepto shims)
- [] Verificar imports correctos: ``grep -r "from core\.config_manager\|from api\.api_utils\|from api\.middlewares" --include="*.py"`` → Debe mostrar muchos
- [] Verificar que no hay imports circulares

Type Hints

- [] Verificar type hints en funciones públicas
- [] Ejecutar ``mypy .`` (si está configurado)
- [] Verificar que no hay ``Any`` innecesarios

Documentación

- [] Todas las funciones públicas tienen docstrings
- [] Docstrings incluyen Args, Returns, Raises
- [] Ejemplos en docstrings donde sea apropiado

Tests

- [] Ejecutar: ``pytest`` → Todos los tests pasan
- [] Cobertura: ``pytest --cov`` → Verificar cobertura

- [] Tests de integración funcionan

■■ Verificación de Arquitectura

Capas

- [] Presentation layer no importa directamente de Domain
- [] Application layer usa ServiceContainer
- [] Domain layer no tiene dependencias de frameworks
- [] Infrastructure implementa contratos de dominio

Estructura

- [] Archivos en ubicaciones correctas según capas
- [] No hay archivos duplicados
- [] Nombres de directorios no conflictúan con Python built-ins

■ Verificación de Calidad

Linting

- [] Ejecutar: `ruff check .` → 0 errores
- [] Ejecutar: `ruff format --check .` → Formato correcto
- [] No hay warnings de deprecación en código nuevo

Performance

- [] No hay imports innecesarios
- [] No hay código muerto
- [] Operaciones I/O optimizadas (donde sea posible)

Seguridad

- [] Inputs validados en todos los endpoints
- [] No hay vulnerabilidades conocidas
- [] Secrets no están hardcodeados

[Nota: El documento original tiene 223 líneas. Se muestra un resumen de las primeras 100 líneas.]